Break, Continue and Loop Fallback

Control a loop mid-flight, and use the else clause as a built-in fallback

The three controls

Statement	Effect
`break`	leave the loop **now**; skip everything remaining
`continue`	skip the rest of **this** pass; jump to the next one
`pass`	do nothing: an empty-body placeholder
`else`	runs when the loop ends **without** a `break`

continue: skip this pass

```
for n in range(1, 8):
    if n % 2 == 0:
        continue          # skip evens
    print(n, end=" ")
```

OUTPUT

```
1 3 5 7
```

Search with a fallback

```
def find(users, name):
    for u in users:
        if u == name:
            return f"{u} found"
    return f"{name} not found"
print(find(["ann", "bob"], "bob"))
print(find(["ann", "bob"], "cy"))
```

OUTPUT

```
bob found
cy not found
```

Retry with a limit

```
for attempt in range(1, 4):
    if attempt == 3:      # pretend 3rd works
        print("connected on", attempt)
        break
    print("retry", attempt)
else:
    print("gave up")
```

OUTPUT

```
retry 1
retry 2
connected on 3
```

Nested loops: break leaves only one level

Use a flag, or move the loops into a function and `return`.

```
found = False
for r in range(3):
    for c in range(3):
        if r * c == 2:
            found = True
            break         # leaves inner only
    if found:
        break             # leaves outer
print(r, c)
```

OUTPUT

```
1 2
```

break: stop early

```
for n in [3, 8, 12, 5]:
    if n > 10:
        print("found", n)
        break
    print("checked", n)
```

OUTPUT

```
checked 3
checked 8
found 12
```

Loop else: the fallback

The `else` block runs only if the loop finished **naturally**. A `break` skips it. Read it as "no break".

```
for n in [3, 8, 12]:
    if n > 20:
        print("found", n)
        break
else:
    print("nothing above 20")
```

OUTPUT

```
nothing above 20
```

Prime check with loop else

```
for n in [7, 9]:
    for d in range(2, n):
        if n % d == 0:
            print(n, "is not prime")
            break
    else:
        print(n, "is prime")
```

OUTPUT

```
7 is prime
9 is not prime
```

while also has else

```
n = 0
while n < 3:
    n += 1
else:
    print("done, n =", n)
```

OUTPUT

```
done, n = 3
```

Mistakes to avoid

Mistake	Why it bites
`continue` before the update in `while`	skips `i += 1`: infinite loop
expecting `else` after a skipped loop	empty loop: `else` **still runs**
`else` lined up with the `if`	then it is **not** loop-else

REMEMBER

- `break` exits the nearest loop; `continue` jumps to its next iteration.
- Loop `else` = "no break happened": use it for "searched everything, found nothing".
- An empty loop (nothing to iterate) also runs its `else`.
- In `while` loops, make sure `continue` does not jump over the counter update.
- To escape nested loops, use a flag or put the loops in a function and `return`.
- Use `pass` as a placeholder for a body you will write later.